Agentic AI 4: Protocols — MCP, A2A, AG-UI

Course syllabus · version 1 · approved 2026-10-11

Who it's for: Developers who have done tracks 1–3 of the Agentic AI roadmap (or can write a tool-calling agent loop, use agent patterns, memory and RAG, and build with LangGraph plus one vendor SDK). Comfortable with Python async, JSON and HTTP; basic Docker helps for Parts 6–8.

What you'll be able to do: Connect agents to tools and data with MCP (2026-07-28 stateless revision, mcp 2.x), to other agents with A2A v1.0, and to users' screens with AG-UI 1.0: build servers and clients for each from the raw wire format up, drive them with a real model, handle elicitation, long-running Tasks, streaming, push notifications and human approval; decide when a capability is an MCP tool and when it is an A2A agent; secure them with OAuth 2.1 (protected resource metadata, audiences, scopes, CIMD, no token passthrough) against tool poisoning, prompt injection through tool results and confused-deputy attacks; and run them across an organisation with one authorization server, agent identities, enterprise-managed authorization (ID-JAG), signed Agent Cards, tenants, a gateway, a private registry and an audit trail.

Running example — Trackr, the team ops desk: Everyone understands issues, teams and severities. Every protocol feature has a natural use: a destructive close_issue (annotations, confirmation), a long export (Tasks), 'close 40 issues?' (elicitation), classifying and fixing (A2A agents), approving a change (AG-UI interrupt), malicious issue text (prompt injection through tool results), and many teams (tenants, gateway, registry). It matches the source's capstone (ops copilot with triage and fix agents).

How it grows: L1: 20 in-memory issues behind a one-tool stdio MCP server called by hand -> L2: tools, runbook resources and a triage prompt over stdio and Streamable HTTP; triage-agent over A2A; a live AG-UI chat -> L3: elicitation, a long export as a Task, OAuth scopes and two different clients (Build 1); orchestrator delegating to triage-agent and fix-agent with human approval (Build 2) -> L4: 4 workers, tool-catalog token budget, binding choices, the MCP-or-A2A exit test -> L5: tests, one trace across AG-UI/A2A/MCP, poisoned servers and injected issues blocked, deployed behind nginx -> L6: one Keycloak for the org, agent identities, ID-JAG, signed cards, many teams as tenants, a gateway with per-tool policy, a private registry, an audit log -> capstone ops copilot.

67 lessons · about 50 hours · 8 parts

Choose your path

Nothing is left out: Complete is the whole course. Shorter paths take the most-used lessons first; each works on its own.

Path	Lessons	Time	Includes
Essentials	23	~17 h	Core lessons: what most real work uses
Professional	53	~42 h	Core + Professional: build, ship and run it in production
Complete	67	~50 h	Everything, including internals, enterprise scale and the frontier

Or as three courses by difficulty

Course	Levels	Lessons	Time
Basic	L1 Foundations + L2 Core	00–11 (19)	~12 h
Intermediate	L3 Practical + L4 Advanced	12–26 (21)	~16 h
Advanced	L5 Production + L6 Enterprise	27–44 (27)	~23 h

LLM use: LLM-centric and LLM-based track (user decision, firm): a real model drives every example where an agent or model acts, core and L1 lessons included, recorded with llm_replay so every lesson runs without a key. No scripted dummy models; the only fake is the in-memory MCP Client in lesson 27, where testing with it is the skill. Protocol mechanics are first shown by hand (raw JSON-RPC over stdio and HTTP, a hand-written Agent Card, raw AG-UI events) because that is the protocol, not a fake model. Use real, standard libraries, not course-made helpers: models from LangChain init_chat_model with bind_tools / AIMessage.tool_calls / ToolMessage; MCP tools reach the model through langchain.mcp.MCPAdapter (langchain-mcp-adapters is archived and capped at mcp<2); official mcp 2.x (MCPServer, Client), a2a-sdk 1.x and

ag-ui-protocol for the protocols. The only course helper is llm_setup.get_llm(<lesson>), which lists every provider and model with key status and recordings, starts llm_replay with one recording per model, and returns the LangChain model; learners switch model in .env or with get_llm(..., provider=, model=). Lessons that teach one vendor's own MCP client (14, 17) call that SDK directly. Servers that call a model (A2A executors, AG-UI endpoints) run in a thread inside the notebook process so their calls are recorded; local HTTP to your own servers is never recorded (ignore_localhost), so a cassette can't stand in for a broken server; the Vertex project id is normalised in cassettes. Prose never depends on one model's exact output; random ids are not asserted on. Recordings: Gemini while building (GOOGLE_CLOUD_API_KEY), plus Claude/OpenAI when those keys are present; never invented. Low cost: every example makes at most 2 model calls (MCP = tool-choice call + answer call; A2A = 1 call in the remote agent + 1 in the orchestrator); a lesson's real-call budget is its llm_calls value (default 4), enforced by llm_replay; checkpoints and the capstone go up to 20 with the cheapest model. Replay makes no calls. Model calls are recorded once and replayed, so learners need no API key.

Tools used

Tool	Role	Lessons
LangChain init_chat_model (Gemini, Claude, OpenAI, Ollama) + llm_replay via llm_setup.get_llm	the real model wherever an agent acts; learner picks any provider, key status shown; one recording per model	most (all but 01, 07b, 21a, 23, 27, 31, 31a, 40a, 43, 43a)
mcp 2.x (MCPServer, Client, TokenVerifier/AuthSettings, OAuthClientProvider, Resolve/Elicit)	official MCP Python SDK for servers, clients and auth	01-07b, 11-17, 23-25a, 27-44
langchain.mcp.MCPAdapter (langchain[mcp] 1.4, beta)	MCP tools into a LangChain model with bind_tools (replaces the archived langchain-mcp-adapters)	01a onward
a2a-sdk 1.x (Starlette/uvicorn, protobuf types)	A2A servers, clients, signed cards	08-09a, 11, 18, 21a-23a, 25-26, 28, 32-32a, 37-39, 44
ag-ui-protocol 1.0 + FastAPI; ag-ui-langgraph	AG-UI event endpoint, interrupts and resume	10-11, 19-20, 22-23a, 25, 28, 31b, 44
OpenAI Agents SDK / Anthropic MCP helpers; hosted MCP connectors	a second, vendor MCP client; hosted connectors read-along	14, 17
Authlib (in-process dev authorization server), PyJWT	local OAuth flows and token minting with no account	15-17, 31-31a, 34-36, 38a
Keycloak 26.8 (Docker)	the organisation's authorization server, EMA (experimental)	34, 36, 39, 44
MCP Inspector 2.x (npx, Node >= 22.19), a2a-cli	debugging and CI checks	27, 32a
agentgateway (binary/Docker; optional kind + Gateway API)	MCP/A2A gateway	40, 40a, 42a, 44
OpenTelemetry SDK, pytest + anyio	tracing and tests	27-28, 33
nginx + Docker Compose	production-style deployment	32-33

Lessons

Priority: **Core** = Essentials path · **Pro** = Professional path · **Deep** = Complete path. ★ = checkpoint project.

Part 1 · L1 Foundations: Why agents need protocols

#		Lesson	You will be able to	Needs	Min
00	Core	**Why agents need protocols: the big picture in plain words**	In very plain language, explain why agents need shared protocols: what an agent has to talk to (tools and data, other agents, people on a screen); where protocols sit in the agentic AI spectrum, from a single chatbot to tool-using agents to teams of agents across companies; what goes wrong without them (custom glue for every pair, M×N integrations, broken or unsafe connections, agents that can't find or trust each other, users who can't see or stop what an agent does); and how MCP, A2A and AG-UI each fix one of these communication lines, who governs them, and what you will build. Set up your model and see which keys you have.	llm	35
01	Core	**JSON-RPC 2.0 and MCP by hand**	Send raw JSON-RPC 2.0 messages over stdio to a tiny MCP server and read the replies: server/discover, tools/list and tools/call with their _meta, plus a result, an error and a notification.	—	30
01a	Core	**Hello MCP: a real model uses your server**	Hand the server's tools to a real model with the standard API — langchain.mcp.MCPAdapter, bind_tools, ToolMessage — and trace which JSON-RPC messages from lesson 01 happen underneath.	llm	25
02	Core	**Checkpoint: your first MCP server, driven by a real model ★**	Build a two-tool Trackr MCP server, check it by raw JSON-RPC, and let a real model answer questions through MCPAdapter.	llm	60

Part 2 · L2 Core: Core primitives: MCP, then A2A, then AG-UI

#		Lesson	You will be able to	Needs	Min
03	Core	**MCP tools in depth**	Define MCP tools correctly: names, titles, descriptions, inputSchema from type hints (JSON Schema 2020-12), outputSchema with structuredContent, annotations, and tool-execution errors (isError) vs protocol errors.	llm	35
04	Pro	**Resources and prompts: context the app and the user control**	Expose read-only data as resources (static URIs and RFC 6570 templates such as runbook://{team}) and reusable prompts with argument completion and pagination, and explain who controls each primitive.	llm	30
05	Core	**The Python MCP client and several servers**	Use mcp 2.x Client (stdio and in-process server object) to list and call tools, then connect a model to two servers at once with MCPAdapter's {"mcpServers": …} config and namespaced tool names.	llm	30
05a	Core	**Wiring servers into real hosts: Claude Desktop, Claude Code, VS Code, Cursor**	Register a local and a remote Trackr server in the config files real hosts read (claude_desktop_config.json, Claude Code .mcp.json, VS Code mcp.json, Cursor mcp.json) and reuse the same mcpServers block in your own client.	llm	25
05b	Core	**stdio done right: stdout, environment and process lifetime**	Avoid the classic stdio failures: a print() on stdout corrupting the JSON-RPC stream, secrets missing because the subprocess gets a minimal environment (pass env=), and orphan server processes after the host exits.	llm	25

#		Lesson	You will be able to	Needs	Min
06	Core	**Remote servers: the Streamable HTTP transport**	Run an MCP server on 127.0.0.1 over Streamable HTTP and talk to it by hand with httpx (single POST endpoint, Accept header, Mcp-Method / Mcp-Name / Mcp-Param headers, HeaderMismatch), then with Client and the model; apply Origin validation and localhost binding.	llm	35
07	Core	**The stateless core of MCP 2026-07-28**	Observe what 'stateless' means: version and capabilities ride on every request in _meta, server/discover replaces the handshake, any worker can answer any request, and state lives in explicit handles.	llm	30
07a	Pro	**Talking to legacy servers (2025-11-25 and older)**	Connect to servers that still use the initialize handshake and Mcp-Session-Id: let Client probe the era, pin mode="legacy" when needed, handle UnsupportedProtocolVersionError, and serve legacy clients from your own server (stateless_http, sticky sessions).	llm, download	35
07b	Pro	**Change notifications: subscriptions/listen**	Open a subscriptions/listen stream, opt in to toolsListChanged, resourcesListChanged and resource subscriptions, read the ack and subscriptionId, and re-listen after the stream drops (no resumption).	—	30
08	Core	**Your first A2A agent: Agent Card, executor and client**	Write an Agent Card by hand, serve it at /.well-known/agent-card.json, implement an AgentExecutor backed by a real model, and call it with the a2a-sdk 1.x client; explain messages, parts, artifacts and opacity.	llm	35
09	Pro	**A2A tasks: lifecycle, multi-turn and streaming**	Drive a task through the v1.0 TASK_STATE_* states (SUBMITTED, WORKING, INPUT_REQUIRED, AUTH_REQUIRED, COMPLETED, FAILED, CANCELED, REJECTED), continue it with contextId/taskId, follow it by polling (GetTask, ListTasks) and streaming (SendStreamingMessage, SubscribeToTask), cancel it, and read errors (google.rpc.Status + ErrorInfo) and the A2A-Extensions header.	llm	40
09a	Deep	**A2A push notifications and webhook security**	Register push-notification configs for long tasks, receive StreamResponse payloads on a local webhook, authenticate and verify them, block SSRF with the SDK's push_url_validator, and handle delivery retries and duplicates.	llm	35
10	Core	**AG-UI: streaming an agent to the user's screen**	Build an AG-UI endpoint that turns a real streaming model run into AG-UI events (RUN_STARTED, TEXT_MESSAGE_*, TOOL_CALL_*, RUN_FINISHED / RUN_ERROR) over HTTP+SSE, and consume it from Python.	llm	35
10a	Deep	**AG-UI state, frontend tools, snapshots and reasoning**	Use the rest of the AG-UI event set: shared state (STATE_SNAPSHOT, STATE_DELTA as JSON Patch), MESSAGES_SNAPSHOT, frontend tools defined by the client in RunAgentInput.tools, and reasoning and activity events.	llm	35
11	Core	**Checkpoint: Trackr v1 — an MCP server, an A2A agent and a live UI ★**	Ship Trackr v1: the MCP server over stdio and Streamable HTTP, triage-agent over A2A, and an AG-UI chat endpoint whose model uses the MCP tools, all verified from Python.	llm	90

Part 3 · L3 Practical: MCP in real apps (Build, part 1)

#		Lesson	You will be able to	Needs	Min
12	Pro	**Asking the user mid-call: elicitation with Multi Round-Trip Requests**	Make a tool pause for user input with MRTR: InputRequiredResult (resultType input_required, inputRequests, requestState), the client's retry with inputResponses and the echoed, integrity-protected requestState; form vs URL elicitation. Roots, sampling and logging appear only as deprecated features you will see in old servers.	llm	35
13	Pro	**Extensions, progress and cancellation**	Negotiate an opt-in extension (vendor-prefixed id in capabilities, off by default, graceful degradation), report progress from a slow tool, and cancel it by closing the response stream.	llm	30
13a	Pro	**Long-running work: the MCP Tasks extension**	Run long work as a task with io.modelcontextprotocol/tasks (CreateTaskResult, tasks/get, tasks/update, tasks/cancel, ttlMs, pollIntervalMs, notifications/tasks over subscriptions/listen) and compare MCP Tasks with A2A tasks.	llm	35
14	Core	**One server, many clients: vendor SDKs and hosted MCP**	Connect Trackr to a second client — the OpenAI Agents SDK (MCPServerStreamableHttp) or Anthropic async_mcp_tool — and know the hosted options (Anthropic MCP connector, OpenAI Responses type 'mcp' with approvals, Gemini Interactions mcp_server) and their limits (public HTTPS, no stdio).	llm, cloud	35
15	Core	**Protecting an MCP server: OAuth 2.1 resource server**	Turn Trackr into an OAuth 2.1 resource server: TokenVerifier and AuthSettings, Protected Resource Metadata (RFC 9728), the 401 WWW-Authenticate challenge, audience validation (validate_token_resource on), and scopes issues:read / issues:write checked per tool.	llm	35
16	Core	**Getting a token: the MCP OAuth client flow and step-up**	Run the client side against a local in-process authorization server (Authlib): PRM -> AS metadata discovery, Client ID Metadata Documents (replacing deprecated DCR), authorization code + PKCE S256 + resource indicator, iss validation, token storage with OAuthClientProvider, and scope step-up after 403 insufficient_scope.	llm	40
17	Core	**Checkpoint (Build, part 1): an MCP server with OAuth and two clients ★**	Deliver the source's Build, part 1: an MCP server over Streamable HTTP for Trackr — or a real system you use (a SQLite database, or GitHub with your own token) — protected by OAuth with read/write scopes and connected to two different clients.	llm	120

Part 4 · L3 Practical: Agents, users and the rest of the stack (Build, part 2)

#		Lesson	You will be able to	Needs	Min
18	Core	**Delegating over A2A: when to call a tool and when to hand off**	Build an orchestrator that uses MCP tools for what it controls and delegates to a remote A2A agent for what it does not, and explain the tool-vs-agent decision.	llm	35
19	Pro	**Human approval over AG-UI: interrupts and resume**	Stop a run for a human decision with an interrupt outcome on RUN_FINISHED and resume on the same thread with resolved or cancelled, including from LangGraph via ag-ui-langgraph (emit_interrupt_outcome).	llm	35
20	Deep	**Rich UIs: MCP Apps and generative UI**	Serve an interactive ui:// resource from a tool with the MCP Apps extension (text/html;profile=mcp-app, sandboxed iframe, CSP) and see where A2UI and AG-UI generative-UI specs fit.	llm, download	35
21	Pro	**AGENTS.md and Agent Skills**	Write an AGENTS.md and an Agent Skill (SKILL.md frontmatter, progressive disclosure) for Trackr, validate it with skills-ref, and serve skills with the Skills over MCP extension.	llm	30
21a	Deep	**Neighbouring standards: AP2 and AGNTCY**	Place AP2 (agent payments: checkout and payment mandates, open vs closed, human-present vs not-present; v0.2, now at the FIDO Alliance) and AGNTCY (Agent Directory/OASF, identity, SLIM) relative to A2A and MCP.	—	30
22	Pro	**Checkpoint (Build, part 2): two A2A agents and a delegating orchestrator ★**	Deliver the source's Build, part 2: triage-agent and fix-agent as A2A servers, an orchestrator that delegates from one to the other, and human approval over AG-UI before any change.	llm	120

Part 5 · L4 Advanced: Trade-offs and internals on one machine

#		Lesson	You will be able to	Needs	Min
23	Deep	**Scaling stateless MCP on one machine**	Measure Trackr over 1 vs 4 uvicorn workers; show that MRTR needs shared RequestStateSecurity keys (and a rotation with an overlap window and replay limit) and that subscriptions across workers need a shared SubscriptionBus; read the transport limits (timeouts, 4 MiB bodies, 1 MiB SSE events).	slow	40
23a	Pro	**Retries, idempotency and deadlines across protocols**	Make retries safe when nothing resumes: re-issued tools/call with an idempotency key, AG-UI retry = new run, A2A messageId dedupe, and one deadline propagated AG-UI -> A2A -> MCP.	llm	35
24	Pro	**Tool catalogs: token cost, selection accuracy and name collisions**	Measure prompt tokens and tool-selection accuracy at 5, 20 and 60 tools, and see tool-name collisions when servers are federated; fix with short descriptions and namespacing.	llm	40
24a	Deep	**Caching: ttlMs, cacheScope and prompt-cache-friendly order**	Honour ttlMs and cacheScope on list/read results, invalidate on listChanged, and keep tool order deterministic so the model provider's prompt cache hits.	llm	30
24b	Deep	**Progressive discovery, Code Mode and response projection**	Replace a huge catalog with a 4-tool discovery facade (discover_server, discover_tools, get_tool_schema, invoke_tool), let the model write code against tools (Code Mode), and trim results with response projection; measure the token savings.	llm	35

#		Lesson	You will be able to	Needs	Min
25	Deep	**Wire bindings compared: A2A JSON-RPC, REST, gRPC; AG-UI SSE vs Protobuf**	Choose a binding from the Agent Card's supportedInterfaces and measure latency and payload size of A2A's JSON-RPC, HTTP+JSON and gRPC bindings and AG-UI's HTTP+SSE vs HTTP+Protobuf.	llm	35
25a	Deep	**MCP server frameworks: MCPServer, FastMCP 4, generated from OpenAPI**	Build the same Trackr tools with the official MCPServer, FastMCP 4 and FastMCP's from_openapi, and compare dependencies, features and the tools/list each produces.	llm	35
26	Pro	**Checkpoint (Exit test): MCP tool or A2A agent? ★**	Pass the source's Exit test: for six Trackr capabilities decide on a whiteboard whether each is an MCP tool (or MCP Task) or an A2A agent, justify with the criteria and your measurements, then implement the two most borderline ones both ways and compare.	llm	90

Part 6 · L5 Production: Shipping protocol services safely

#		Lesson	You will be able to	Needs	Min
27	Core	**Testing and debugging: in-memory tests and MCP Inspector**	Test Trackr with the in-process Client and pytest, snapshot tools/list, debug with MCP Inspector 2.x (web, --cli, --tui), know the conformance suite and SDK tiers, and check A2A cards with a2a-cli.	download	35
27a	Pro	**Tool-selection evals with recorded model calls**	Evaluate whether the model picks the right tool and arguments for a fixed prompt set, recording model calls but keeping your own local servers live (ignore_localhost), and keep tool descriptions under version control.	llm	35
28	Pro	**Observability: one trace across AG-UI, A2A and MCP**	Propagate one OpenTelemetry trace from an AG-UI run through the orchestrator, A2A and MCP (_meta traceparent), log to stderr, meter per tool from Mcp-Method / Mcp-Name headers, and measure tracing overhead.	llm	35
29	Core	**Malicious servers: tool poisoning, rug pulls and shadowing**	Show a real model steered by hidden instructions in a tool description, a description changed after approval, and one server's text shadowing another's tool; defend with full-description review, pinned hashes of tools/list, namespacing, treating annotations as untrusted (and possibly deprecated), and confirmation for destructive tools.	llm	40
29a	Pro	**Containing local servers: sandboxing stdio servers**	Run an untrusted stdio server in a container with a minimal environment, read-only filesystem and no or allow-listed egress, and see why: stdio servers run with your privileges (2026 STDIO command-execution CVEs).	llm, docker	35
30	Core	**Prompt injection through tool results and the lethal trifecta**	Reproduce an indirect prompt injection where a malicious issue body makes the agent leak private data, explain the lethal trifecta, and break it by design: split capabilities, allow-lists, confirmation gates and redaction.	llm	40
31	Core	**Token passthrough and the confused deputy**	Show why forwarding a user's token upstream is forbidden, and how a confused-deputy attack works on an MCP OAuth proxy with a static client id; fix with audience checks, per-client consent, exact redirect-URI match, single-use state and __Host- cookies.	—	40

#		Lesson	You will be able to	Needs	Min
31a	Deep	**Client-side OAuth attacks: SSRF, malicious authorization URLs, mix-up**	Harden an MCP client against SSRF during metadata discovery (private-range block-list, HTTPS only), malicious authorization_endpoint URLs (CVE-2025-6514 in mcp-remote) and authorization-server mix-up (iss).	—	30
31b	Pro	**Securing the AG-UI endpoint**	Protect the AG-UI endpoint: authenticate the user, set CORS and CSRF defences, and treat client-sent state, messages, tools and forwardedProps as untrusted input.	llm	30
32	Pro	**Deploying protocol services**	Package Trackr's MCP, A2A and AG-UI services: ASGI mount into FastAPI, health route, proxy headers behind TLS, allowed hosts/origins, X-Accel-Buffering for SSE, a container image; for A2A a durable task store with TTL/garbage collection and multi-replica streaming.	llm, docker	40
32a	Pro	**Releasing changes: protocol revisions and tool descriptions**	Roll out a protocol revision without breaking clients (dual-stack legacy route, deprecation windows, A2A-Version and the a2a-sdk 0.3 compat mode, rotating requestState keys during a rolling deploy) and ship tool-description changes like code: eval-gated, canaried, reversible.	llm	40
33	Core	**Checkpoint: production-ready Trackr ★**	Harden Trackr for one team: tests and Inspector checks, pinned tool hashes, injection defences, correct OAuth (audiences, scopes, no passthrough), with a production checklist. Professional path adds tracing, sandboxing, AG-UI endpoint security and the container deployment.	llm	120

Part 7 · L6 Enterprise: Identity and authorization across an organisation

#		Lesson	You will be able to	Needs	Min
34	Pro	**One authorization server for the organisation: Keycloak**	Run Keycloak 26.8 in Docker as the organisation's authorization server for two MCP servers: realm, client scopes, audience per resource, CIMD and resource indicators, token lifetimes and refresh rotation, and encrypted token storage on the client.	llm, docker	40
35	Pro	**Agent identity: client credentials and token exchange**	Give an unattended agent its own identity with the client-credentials extension (ClientCredentialsOAuthProvider) and let a server call an upstream API on a user's behalf with RFC 8693 token exchange, auditing 'agent X on behalf of user Y'; know the roadmap (DPoP, workload identity federation, WIMSE).	llm	40
36	Deep	**Enterprise-managed authorization (ID-JAG)**	Implement the Enterprise-Managed Authorization flow: sign in to the enterprise IdP, exchange the ID token for an Identity Assertion JWT Authorization Grant, redeem it at the MCP authorization server — no per-server consent, central revocation.	llm, docker	40
37	Pro	**Securing A2A: auth schemes, extended cards and signed Agent Cards**	Protect triage-agent with securitySchemes and per-skill requirements, serve an authenticated extended card, sign the Agent Card (JWS over RFC 8785 JSON) and verify it with a restricted algorithm list so a spoofed card is rejected.	llm	40
38	Pro	**Multi-tenant MCP servers and A2A agents**	Serve many teams from one Trackr: tenant from a verified claim, queries and handles bound to tenant and user, the A2A tenant field, no existence leaks, per-tenant quotas against noisy neighbours, and per-tenant encryption of the A2A task store; learn from the Asana MCP cross-tenant leak.	llm	40
38a	Pro	**Multi-hop delegation: chained token exchange**	Carry the user's authority across user -> orchestrator -> A2A agent -> MCP server with chained RFC 8693 token exchange and act (actor) claims, narrowing scope and audience at each hop and auditing every hop.	llm	35
39	Pro	**Checkpoint: organisation-wide authorization for Trackr ★**	Run Trackr for several teams under one Keycloak: user and agent identities, per-resource audiences, step-up, multi-hop delegation, signed A2A cards and tenant isolation, with an access-review table.	llm, docker	120

Part 8 · L6 Enterprise: Gateways, registries and governance

#		Lesson	You will be able to	Needs	Min
40	Pro	**An MCP and A2A gateway: central auth and per-tool policy**	Put agentgateway in front of Trackr and the runbooks server: JWT validation, per-tool RBAC with CEL, rate limits, federation into one endpoint, response redaction, metrics keyed on Mcp-Method / Mcp-Name; optional kind + Gateway API section.	llm, docker, download, k8s	40
40a	Pro	**Gateway failure modes and overhead, measured**	Measure the gateway's p50/p99 overhead and rate-limit behaviour, then break things: upstream server down, circuit breaking, slow upstream, large bodies and long SSE streams through the proxy.	docker, slow	40

#		Lesson	You will be able to	Needs	Min
41	Deep	**A private MCP registry and server admission**	Run a private registry using the MCP Registry's server.json shape (reverse-DNS names, namespace ownership, versions), gate admission with a description scan and pinned hashes, start new tools disabled until an owner enables them, label read-only vs destructive (annotations are hints and may be deprecated), and publish Server Cards / an AI Catalog.	llm, cloud	40
42	Pro	**Audit trail and compliance for agent protocols**	Design an audit log for protocol traffic (user, agent, on-behalf-of chain, tool, args hash, result size, trace id), encrypt and retain it, and meet residency and deletion obligations (e.g. the Anthropic MCP connector is not ZDR-eligible; OpenAI hosted MCP is ZDR/residency compatible but third-party servers follow their own policy).	llm	35
42a	Pro	**Incident response: kill switch, drills and the threat landscape**	Run an incident drill: detect a compromised server from the audit log, disable it at the gateway in one call, list affected runs by trace id; map defences to the OWASP Top 10 for Agentic Applications and the MCP CVE timeline (only ids verified in research; others marked to check).	llm	35
43	Deep	**Case studies: org-wide agent-protocol platforms**	Use published write-ups (Uber MCP Gateway: 800+ servers, 5,000+ tools; Block goose; Cloudflare's stateless MCP; AWS MCP Gateway & Registry; A2A adoption from the LF release) to design a platform for 5,000 engineers, with a capacity model built from your own measurements.	—	40
43a	Deep	**The protocol frontier and how to track it**	Follow where the protocols are going and how to track them: SEPs and working groups, conformance results, HTTP over stdio, DPoP and workload identity, progressive discovery, Server Cards, A2A in AAIF, AG-UI 1.x.	cloud	30
44	Pro	**Capstone: the enterprise ops copilot ★**	Build the source's capstone preview: MCP servers for Trackr, a git repository and a SQLite database behind the gateway with org OAuth; an orchestrator delegating over signed A2A to triage-agent and fix-agent with chained token exchange; AG-UI approval before any change; traced, audited and multi-tenant.	llm, docker, slow	240

Enterprise depth coverage

Checklist item	Lessons
1 distributed architecture internals	N/A — covered in track agentic-ai-5-multi-agent; stateless MCP vs sessionful servers previewed in lessons 07 and 23
2 cluster operations hands-on	N/A — covered in track agentic-ai-6-production; multi-worker stateless MCP in lesson 23
3 failure and recovery	N/A — covered in track agentic-ai-6-production; protocol retry/idempotency in 23a and gateway failure modes in 40a
4 extreme scale	N/A — covered in track agentic-ai-6-production; tool-catalog cost at scale in lessons 24, 24b and the capacity model in 43

Checklist item	Lessons
5 kubernetes and infrastructure	N/A — covered in track agentic-ai-6-production; optional agentgateway-on-kind section in lesson 40
6 performance engineering at scale	N/A — covered in track agentic-ai-6-production; single-machine measurements in lessons 23, 25, 40a
7 high availability and disaster recovery	N/A — covered in track agentic-ai-6-production; durable multi-replica A2A task store in lesson 32
8 zero-downtime change	N/A — covered in track agentic-ai-6-production; protocol-revision dual-stack rollout and the tool-description release pipeline (description change = behaviour change; eval-gated, canaried) in lesson 32a
9 multi-tenancy and isolation	N/A — covered in track agentic-ai-6-production; protocol-level tenant isolation and per-tenant quotas in lesson 38
10 security and compliance at org scale	15, 16, 29, 29a, 30, 31, 31a, 31b, 34, 35, 36, 37, 38, 38a, 39, 40, 40a, 41, 42, 42a
10 security — encryption at rest (client TokenStorage 34, A2A task stores 38, audit log 42)	34, 38, 42
11 cost engineering	N/A — covered in track agentic-ai-6-production; tool-catalog token levers in lessons 24, 24a, 24b
12 platform and organisation patterns	N/A — covered in track agentic-ai-6-production; the protocol side (gateway, private registry, platform design) in lessons 40, 41, 43
13 frontier	N/A — covered in track agentic-ai-9-frontier; protocol roadmap and how to track it in lesson 43a
14 case studies	N/A — covered in track agentic-ai-6-production; protocol case studies (Uber, Block, Cloudflare, AWS) in lesson 43

Your syllabus → lessons

Source item	Lessons
Level 4 › Goal: three interfaces of an enterprise agent (MCP tools/data, A2A agents, AG-UI screen)	00, 11
Level 4 › MCP and A2A are complementary, not rivals	00, 18, 26
Level 4 › MCP › Current spec 2026-07-28, stateless protocol core	07, 07b, 23
Level 4 › MCP › MCP is a Linux Foundation project (corrected: an LF Projects series hosted by the Agentic AI Foundation since 2025-12-09; individual maintainers)	00, 43
Level 4 › MCP › Architecture: host, client, server; JSON-RPC 2.0; capability and version negotiation (corrected: no handshake — per-request _meta and server/discover)	01, 01a, 05, 05a, 07, 07a
Level 4 › MCP › Server primitives: tools, resources, prompts	03, 04
Level 4 › MCP › Client primitives: sampling, elicitation, roots (corrected: elicitation via Multi Round-Trip Requests; roots/sampling/logging deprecated, taught as 'old servers')	12
Level 4 › MCP › Transports: stdio for local, Streamable HTTP for remote (corrected: POST-only, no sessions or resumability; HTTP+SSE deprecated)	01, 05b, 06

Source item	Lessons
Level 4 › MCP › Authorization: OAuth 2.1, protected resource metadata, scoped tokens, no token passthrough (added: CIMD replaces deprecated DCR, iss validation, step-up, ID-JAG)	15, 16, 31, 34, 35, 36, 38a
Level 4 › MCP › Extensions framework: Tasks (long-running work) and MCP Apps (interactive UI)	13, 13a, 20
Level 4 › MCP › The MCP Registry (corrected: preview, public servers only — private needs your own registry)	41
Level 4 › MCP › The MCP Inspector for debugging	27
Level 4 › MCP › Security: tool poisoning, prompt injection through tool results, confused deputy, over-broad scopes	15, 16, 29, 29a, 30, 31, 31a
Level 4 › A2A › v1.0 (March 2026), first stable release	08
Level 4 › A2A › Governed by the Linux Foundation, 150+ organisations (corrected: AAIF Growth Stage project since 2026-08-27; '150+' is the LF figure of 2026-04-09)	00, 43
Level 4 › A2A › Agent Card at /.well-known/agent-card.json; signed cards for identity	08, 37
Level 4 › A2A › Tasks with a lifecycle (corrected: 8 TASK_STATE_* states incl. CANCELED, REJECTED, AUTH_REQUIRED)	09
Level 4 › A2A › Messages, parts and artifacts (corrected: single Part oneof text|raw|url|data)	08, 09
Level 4 › A2A › Streaming over SSE, push notifications	09, 09a
Level 4 › A2A › JSON-RPC, gRPC and REST bindings	25
Level 4 › A2A › Agents stay opaque	08, 18
Level 4 › A2A › AP2 for agent-initiated payments (corrected: v0.2, donated to the FIDO Alliance 2026-04-28)	21a
Level 4 › AG-UI › Event stream between agent backend and front end (text deltas, tool calls, state updates)	10, 10a
Level 4 › AG-UI › Human approvals (interrupt outcome + resume, AG-UI 1.0)	19
Level 4 › AGENTS.md and Agent Skills	21
Level 4 › AGNTCY: discovery, identity, observability (note: AAIF membership disputed)	21a
Level 4 › Build: MCP server for a real system with OAuth, connected to two different clients	14, 15, 16, 17
Level 4 › Build: wrap two agents as A2A servers, one delegates to the other	18, 22
Level 4 › Exit test: whiteboard — MCP tool versus A2A agent	26
Related capstone (track 9): enterprise ops copilot — MCP servers, A2A triage and fix agents, human approval	44
Specs named: MCP spec 2026-07-28 + release notes, MCP roadmap (Aug 2026), A2A spec, A2A one-year update, LF A2A announcement	00, 07, 43, 43a

Out of scope

The agent loop, tool calling basics, structured output, LangGraph and vendor SDK basics — Tracks agentic-ai-1-foundations to agentic-ai-3 (prerequisites); used here, not re-taught.

Multi-agent orchestration patterns, durable execution and runtime architecture — Track agentic-ai-5-multi-agent (enterprise item 1); this track teaches only the A2A wire between agents.

Scaling, Kubernetes operations, HA/DR, load testing, FinOps and tenancy budgets for agent runtimes and gateways — Track agentic-ai-6-production (items 2-9, 11, 12, 14); protocol-specific practice is kept here in lessons 23-25, 32, 40-43.

Prompt-injection defence design patterns research (CaMeL, dual-LLM, plan-then-execute, AgentDojo) — Track agentic-ai-7-advanced-agents; lesson 30 teaches the protocol-level attack and capability splitting and points there.

Frontier of agentic AI beyond protocol roadmaps — Track agentic-ai-9-frontier (item 13).

Front-end frameworks for AG-UI (CopilotKit React, Next.js, React Native) and writing your own MCP Apps host — JavaScript front-end work, a separate front-end topic not in this roadmap; AG-UI is taught from Python endpoints and clients, MCP Apps rendered by the reference host.

TypeScript, Go, Java, C# and Rust SDKs — The course is Python; other SDKs are named in lesson 27 (SDK tiers) only.

Self-hosting the official MCP Registry codebase and publishing to the public registry — The official registry is in preview and 'not designed for self-hosting'; publishing needs a public namespace. Lesson 41 builds a private registry and reads the public one.

Commercial gateways and IdPs (Kong, Zuplo, Obot, Prefect Horizon, Okta, Entra, Auth0) hands-on — Paid or platform-bound; the open-source equivalents (agentgateway, Keycloak) are taught and the commercial ones named in lessons 40 and 43.

Older or competing agent protocols (IBM ACP, ANP) — Current status unverified in research; named once in lesson 00 for orientation.

Syllabus history

Version	Date	Changes
1	2026-10-11	first approval